Desafio Técnico - Desenvolvedor Junior

Objetivo

Avaliar seus conhecimentos fundamentais em desenvolvimento backend e banco de dados. Este é um projeto simples e direto para demonstrar que você compreende os conceitos básicos.

O Que Você Precisa Fazer

Criar uma aplicação backend simples que:

- Receba dados via API REST (ou GraphQL)
- Valide e processe esses dados
- Insira os registros em um banco de dados
- Retorne respostas apropriadas

Pronto! É só isso.

Requisitos Técnicos

Flexibilidade Total

- Linguagem: Python, JavaScript, Java, Delphi, Go, Rust, C#... qualquer uma!
- Banco de Dados: PostgreSQL, MySQL, MongoDB, SQLite, Firebase... o que você escolher!
- Use IA se quiser: ChatGPT, Claude, Copilot... você pode pedir ajuda, mas precisa entender cada linha do código (veja abaixo)

O Que Deve Estar no Seu Repositório

```
seu-projeto/
├── README.md (OBRIGATÓRIO)
├── API.md ou DOCUMENTAÇÃO.md (OBRIGATÓRIO)
├── Dockerfile (opcional, mas recomendado)
├── docker-compose.yml (opcional)
├── src/
|   └── seu código aqui
├── requirements.txt (ou package.json, pom.xml, etc)
└── .gitignore
```

Requisitos Obrigatórios

1. README.md

Deve conter:

- O que o projeto faz (descrição breve)
- Como instalar e rodar localmente
- Dependências necessárias
- Como usar a aplicação
- Se usou Docker, as instruções

2. Documentação da API

Detalhe cada endpoint:

- URL da rota
- Método HTTP (GET, POST, PUT, DELETE, etc)
- O que envia (request body, se aplicável)
- O que retorna (response, exemplos)
- Códigos de status HTTP (200, 400, 500, etc)
- Tratamento de erros

Exemplo:

```
POST /usuarios
Descrição: Cria um novo usuário
Request:
{
  "nome": "João Silva",
  "email": "joao@email.com"
}
Response (201):
{
  "id": 1,
  "nome": "João Silva",
  "email": "joao@email.com",
  "criado_em": "2025-04-17"
}
```

3. Código no GitHub

- Repositório público
- Commits com mensagens claras

- Um README explicativo

4. Docker (Opcional, mas Recomendado)

Se incluir Docker:

- Dockerfile funcional
- docker-compose.yml para facilitar
- Isso economiza muito tempo nosso na avaliação

Ideias de Projetos

Lembre-se: simples é melhor. Foque em fazer algo pequeno bem feito.

Opção 1: Sistema de Tarefas (To-Do)

- Criar tarefa
- Listar tarefas
- Marcar como concluída
- Deletar tarefa

Opção 2: Gerenciador de Contatos

- Adicionar contato (nome, telefone, email)
- Listar contatos
- Buscar por nome
- Atualizar contato
- Deletar contato

Opção 3: Blog Simples

- Criar post (título, conteúdo, autor)
- Listar posts
- Buscar posts
- Comentários em posts

Opção 4: Sistema de Produtos

- Adicionar produto (nome, preço, estoque)
- Listar produtos
- Atualizar preço/estoque

- Deletar produto

Opção 5: Sua Criatividade

Tem uma ideia melhor? Use! Criatividade e proatividade ganham consideração diferenciada.

Critérios de Avaliação

Obrigatório

- Funcionamento básico (API responde, dados vão para BD)
- README.md completo
- Documentação da API
- Código no GitHub

Diferenciais

- Docker/Docker Compose configurado
- Validação robusta de dados
- Tratamento de erros apropriado
- Testes automatizados
- Código bem organizado e nomeado
- Criatividade no projeto escolhido
- Funcionalidades extras além do básico
- Git com histórico de commits bem feito

Aviso Importante

Durante a Entrevista, Você Será Questionado Sobre:

1. Cada decisão técnica: Por que escolheu essa linguagem? Esse banco de dados?
2. Como o código funciona: Entendeu cada linha? Consegue explicar o fluxo?
3. Problemas e soluções: Encontrou dificuldades? Como resolveu?
4. Se usou IA: Está tudo bem usar! Mas você precisa entender e conseguir explicar tudo que está ali
5. O que aprendeu: Como esse projeto te desenvolveu?

Resumo: Não é suficiente só copiar e colar. Você deve compreender totalmente o que fez.

Como Entregar

1. Crie um repositório público no GitHub
2. Desenvolva seu projeto com commits significativos
3. Escreva um bom README.md e documentação da API
4. Se usar Docker, teste antes de enviar
5. Compartilhe o link do repositório conosco

Dúvidas Frequentes

- A aplicação precisa ser bonita? Não, funcionalidade vem primeiro.
- Preciso usar um padrão específico? Não, escolha o que faz sentido.
- Posso usar frameworks e bibliotecas? Sim! Use as ferramentas modernas.
- Posso pedir ajuda? Sim, mas entenda o que está fazendo.

Considerações Finais

Menos é mais. Um projeto simples, bem documentado e funcional é muito melhor do que algo complexo que não funciona. Foque em:

1. Código que funciona
2. Documentação clara
3. Estrutura legível
4. Fácil de rodar